

Clase 13

Deployment Render + Aiven

SEMANA 13 - Unidad 7

Docker

Profiles

Variables de entorno

MySQL cloud

Render

Aiven

Docker

<https://...onrender.com>

Objetivos de la Clase

Hoy cursosapp sale de la computadora y va a internet.

- 1 Crear una base de datos MySQL en la nube (Aiven).
- 2 Desplegar la aplicacion en un hosting externo (Render) via Docker.
- 3 Configurar variables de entorno para separar credenciales del codigo.
- 4 Activar un profile de produccion sin tocar el de desarrollo.
- 5 Diagnosticar los errores mas comunes de un primer deploy.

Agenda de la Clase

2 horas y media - hoy con mas configuracion que codigo nuevo.

01

Arquitectura del despliegue

20 min

Por que separar app y base de datos en la nube

02

Lab - Aiven

20 min

Crear la base de datos MySQL en la nube

03

Profiles + Docker

32 min

application-prod.properties y por que Render usa Docker

04

Lab - Render + verificación

65 min

Deploy, variables de entorno, seed, prueba final

Por Que Separar App y Base de Datos

Cada servicio hace UNA cosa bien - y escala por separado.

Render

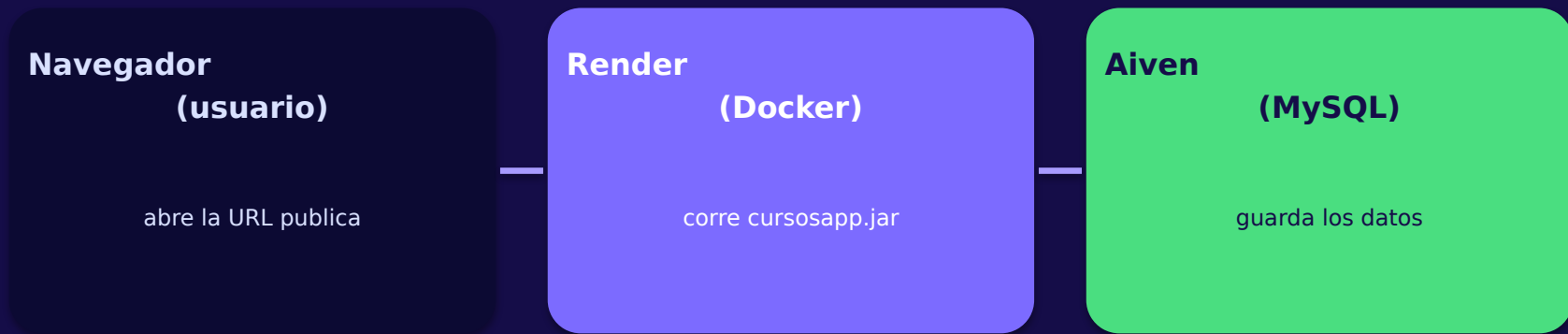
Corre la aplicacion
(el .jar de cursosapp)
Sin estado propio

Aiven

Corre la base de datos
(MySQL gerenciado)
Guarda todos los datos

Arquitectura del Deploy de Hoy

De donde sale cada credencial.



Render -> Aiven se conecta con DB_URL/DB_USERNAME/DB_PASSWORD (variables de entorno). El navegador nunca ve la base de datos directamente.

Que NO Cambia del Proyecto

Cero cambios de codigo Java hoy.



Entidades

Curso, Profesor, Usuario - identicas



Seguridad

Login, roles, @PreAuthorize - identicos



API REST

/api/cursos - identica

Laboratorio - Aiven - base de datos en la nube

Parte A del lab_clase13.md - 20 minutos

Manos a la obra - cuentas, servicios en la nube, variables de entorno

1 Crear cuenta

aiven.io - gratis, sin tarjeta

2 Crear servicio

MySQL, plan Free, cualquier region

3 Esperar 'Running'

puede tardar 1-2 minutos

4 Anotar credenciales

host, puerto, usuario, password, defaultdb (Quick connect)

Profiles de Spring

application.properties + application-prod.properties

Como decide Spring que archivo usar

application.properties	(siempre se carga)
application-prod.properties	(solo si esta activo el profile "prod")

SPRING_PROFILES_ACTIVE=prod -> activa el segundo

Los valores del profile activo pisan a los del archivo base para las claves repetidas. Lo que no se toca en el profile queda igual que siempre.

Las Credenciales NUNCA en el Archivo

`${DB_URL}` es un placeholder, no un valor real.

`application-prod.properties` (se sube a GitHub tal cual)

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

Si escribieras la password real aca...

...cualquiera con acceso al repo de GitHub (o al historial de git, incluso borrandolo despues) tendria la contrasena real de tu base de datos.

El Puerto lo Decide la Plataforma

```
server.port=${PORT:8080}
```

Comparacion local vs. produccion

```
# En tu computadora: siempre 8080  
server.port=8080
```

```
# En produccion: el hosting decide el puerto real  
server.port=${PORT:8080}
```

Render pasa el puerto real en la variable PORT. Si esa variable no existe (ej. corriendo este profile en tu compu por error), cae al 8080 de siempre.

Laboratorio - application-prod.properties

Parte B del lab_clase13.md - 15 minutos

Manos a la obra - cuentas, servicios en la nube, variables de entorno

1 Crear archivo

en src/main/resources/, nombre exacto

2 Copiar contenido

del lab_clase13.md - placeholders con \${...}

3 Commit + push

este archivo SI se sube a GitHub

Por Que Render Necesita Docker para Java

Node/Python/Ruby/Go tienen runtime nativo. Java, no.

Node.js / Python / Ruby / Go

Runtime nativo

Java

Requiere Docker

No es tema de este curso escribir Dockerfiles - el que ya viene armado alcanza para desplegar. Vale entender el motivo: Docker es un formato estandar que cualquier hosting puede correr, sin importar el lenguaje de adentro.

El Dockerfile en 2 Minutos

Build en 2 etapas: compilar, despues correr.

Dockerfile (resumido)

```
FROM eclipse-temurin:25-jdk AS build    # Etapa 1: compilar
RUN ./mvnw clean package -DskipTests

FROM eclipse-temurin:25-jre             # Etapa 2: correr (liviana)
COPY --from=build ../cursosapp.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

La imagen final solo tiene el JRE + el .jar - no el compilador ni el codigo fuente completo. Mas liviana, mas rapida de desplegar.

Laboratorio - Render - conectar repo + variables de entorno

Parciales y el lab_01aseo.md - 25 minutos

Manos a la obra - cuentas, servicios en la nube, variables de entorno

1 Crear cuenta

render.com - gratis, sin tarjeta

2 Web Service

conectar el repo de GitHub, Environment: Docker

3 4 variables

SPRING_PROFILES_ACTIVE, DB_URL, DB_USERNAME, DB_PASSWORD

4 DB_URL

jdbc:mysql://host:puerto/defaultdb?sslMode=REQUIRED

Disparar el Deploy

Demo con un deploy ya hecho de antemano.

Logs esperados en Render

```
Building...  
==> Cloning from GitHub...  
==> Building Docker image...  
==> Deploying...  
==> Your service is live at https://cursosapp-xxxx.onrender.com
```

Laboratorio - Deploy, seed y verificacion

Parte E del lab_clase13.md - 15 minutos

Manos a la obra - cuentas, servicios en la nube, variables de entorno

- 1 Copiar URL publica** la que da Render al terminar el deploy
- 2 Correr seed-data.sql** contra la base de Aiven, no localhost
- 3 Probar en el navegador** home, /cursos, login, /api/cursos
- 4 Postman** cambiar base_url a la URL de Render

Problemas Comunes en Vivo

Revisando el propio deploy de cada uno.

Build falla (Docker)

Dockerfile en la raiz + Environment: Docker

No conecta a la base

DB_URL/USERNAME/PASSWORD bien copiados

Error SSL/handshake

falta ?sslMode=REQUIRED en DB_URL

/cursos vacio

falta correr seed-data.sql contra Aiven

| Que Queda del Curso

S9-S13 fue la materia nueva. Ahora, cierre.

S14

Wrap-up ligero + Caso Practico #2 (asincronico)

S15-S16

Exposiciones + Portafolio (10%) + Final (25%) + Articulo (10%)

Cierre - El Ciclo Completo

Codigo, seguridad, API y ahora deployment real.



Deploy real

URL publica funcionando



Base en la nube

MySQL gerenciado en Aiven



Buenas practicas

variables de entorno, profiles

De aca en mas

Esto es, literalmente, lo que hace un desarrollador full-stack en su trabajo. El resto del curso se enfoca en cerrar el proyecto final.